

Infrastructură Server

Manual tehnic și de reproducere

Torb Logistic SRL
Versiunea 1.0 — Iunie 2026



Cuprins

1. Introducere	4
1.1. Scopul documentului	4
1.2. Cum este organizat	4
1.3. Versiuni și actualizări	4
2. Prezentare generală a infrastructurii	5
2.1. Două medii pe același server	5
2.2. Schema fluxului unei cereri	6
3. Serverul (VPS)	7
3.1. Date de identificare și acces	7
3.2. Conectarea prin SSH	7
3.3. Panoul furnizorului (consola de urgență)	7
4. Sistemul de operare și pachetele instalate	8
4.1. Sistemul de operare	8
4.2. Pachete de sistem necesare	8
4.3. Actualizări automate de securitate	8
4.4. Versiunile exacte instalate	8
5. Utilizatori, permisiuni și proprietatea fișierelor	10
5.1. Utilizatorii relevanți	10
5.2. Proprietatea și permisiunile fișierelor	10
6. Aplicația: medii, servicii și fișiere	11
6.1. Producție	11
6.2. Dezvoltare / Test	11
6.3. Punctul de intrare al aplicației (WSGI)	11
6.4. Serviciile systemd	12
7. nginx, HTTPS și firewall	13
7.1. nginx (reverse proxy)	13
7.2. HTTPS (Let's Encrypt / Certbot)	14
7.3. Firewall (ufw)	14
7.4. fail2ban	15
7.5. Configurarea SSH (sshd) și întărirea recomandată	15
7.6. Configurarea și folosirea cheilor SSH	15
7.6.1. Bootstrap: cum ajunge prima cheie pe server	15
7.6.2. Generarea unei chei (pe calculatorul de administrare — Windows PowerShell)	16
7.6.3. Instalarea cheii publice pe server	16
7.6.4. Conectarea	16
7.6.5. Mai multe calculatoare	16
7.7. Tailscale (rețea privată mesh)	16
8. Variabile de mediu și informații sensibile	17
8.1. Fișierul .env	17
8.2. Inventarul valorilor (FĂRĂ valori aici)	17
8.3. Email / SMTP (resetare parolă)	17
8.4. Unde sunt stocate valorile reale	18

8.5. Seiful de secrete (KeePassXC)	18
9. Implementare automată (CI/CD — GitHub Actions)	20
9.1. Depozitul de cod	20
9.2. Conducta de implementare	20
9.3. Cum funcționează o implementare	20
9.4. Revenirea la versiunea anterioară (rollback)	21
9.5. Dezvoltare locală (Windows)	21
10. Procedura de reconstrucție de la zero (runbook)	22
10.1. Pasul 1 — Sistem de bază și pachete	22
10.2. Pasul 2 — Utilizatori și grupuri	22
10.3. Pasul 3 — SSH (port 2112) și firewall	22
10.4. Pasul 4 — Codul aplicației (prod + dev)	22
10.5. Pasul 5 — Fișierul .env	23
10.6. Pasul 6 — Baza de date și directoarele de backup	23
10.7. Pasul 7 — Serviciile systemd	23
10.8. Pasul 8 — nginx și HTTPS	23
10.9. Pasul 9 — Backup zilnic (cron)	23
10.10. Pasul 10 — OpenClaw (opțional, integrarea AI)	23
10.11. Pasul 11 — Verificare finală	23
11. Stare curentă și acțiuni în așteptare	24
11.1. Lista de verificare după restabilirea accesului sudo	24
11.2. Recomandare pe termen lung	25
11.3. De adăugat: swap	25
12. Baza de date și backup-uri	26
12.1. Baza de date	26
12.2. Sistemul de backup (numai producție)	26
12.3. Niveluri de protecție a datelor	26
13. OpenClaw (integrarea AI)	27
13.1. Cum rulează	27
13.2. Integrarea cu Flask	27
14. Mentenanță de rutină și depanare	28
14.1. Comenzi utile	28
14.2. Probleme frecvente	28
15. Anexa A — Comenzi pentru completarea informațiilor lipsă	29
15.1. Bloc A — fără sudo (rulabil acum, ca openclaw)	29
15.2. Bloc B — necesită sudo (rulabil după revenirea openclaw în grupul sudo)	29
16. Glosar (pentru cititori non-tehnici)	30
17. Contact și suport	30

1. Introducere

Acest document descrie **întreaga infrastructură de server** pe care rulează aplicația Torb Logistic (app.robrands.ro): ce tehnologii sunt folosite, cum sunt configurate, unde se află fișierele importante și — cel mai important — **cum poate fi reconstruită de la zero** pe un alt server.

Manualul este structurat pe două niveluri de lectură:

- **Pentru cititori non-tehnici** (conducere, responsabil de relația cu furnizorul de hosting): capitolele 2 (Prezentare generală) și 11 (Stare curentă), plus **Glosarul** de la final, explică în cuvinte simple ce este serverul, ce face fiecare componentă și ce decizii sunt necesare.
- **Pentru administratori / personal tehnic**: toate capitolele conțin detalii exacte — căi de fișiere, comenzi, parametri de configurare și o procedură pas-cu-pas de reconstrucție (capitolul 10).

1.1. Scopul documentului

1. Să servească drept **sursă unică de adevăr** despre cum este construit serverul.
2. Să permită **reproducerea completă** a infrastructurii pe un server nou, fără cunoștințe prealabile.
3. Să grupeze, alături de manual, **informațiile sensibile** (parole, chei, token-uri) într-un mod sigur — vezi capitolul 9.

1.2. Cum este organizat

Notă privind „sursa de adevăr”: faptele despre server se mențin în fișierul tehnic context/infrastructure.md din depozitul de cod. Acest manual este versiunea citibilă și orientată spre reproducere a acelor fapte. Când o informație se schimbă pe server, actualizați **ambele**.

1.3. Versiuni și actualizări

Versiune	Data	Modificări
1.0	Iunie 2026	Versiune inițială a documentului

2. Prezentare generală a infrastructurii

Întreaga aplicație rulează pe **un singur server virtual (VPS)** închiriat de la un furnizor de hosting. Pe acest server convieșuiesc mai multe componente, fiecare cu un rol clar:

Componentă	Rol, pe înțelesul tuturor
nginx	„Recepționarul,” — primește toate cererile din internet (HTTPS) și le trimite mai departe către aplicație. Tot el se ocupă de certificatul de securitate (lacătul din browser).
Gunicorn	„Motorul,” care rulează efectiv aplicația Python (codul Torb). Rulează în fundal și ascultă numai cereri venite de la nginx.
Flask (aplicația Torb)	Programul propriu-zis — paginile, rapoartele, sincronizările de stoc, modulul de forecast etc.
SQLite	Baza de date — un singur fișier (<code>torb.db</code>) care conține toate datele aplicației.
systemd	„Administratorul de servicii,” al sistemului — pornește automat aplicația la repornirea serverului și o repornește dacă se oprește.
Certbot / Let's Encrypt	Furnizează și reînnoiește automat certificatul HTTPS (gratuit).
ufw + fail2ban	Protecția — firewall (ce porturi sunt deschise) și blocarea automată a atacurilor de tip „ghicire de parolă,” pe SSH.
GitHub Actions	„Banda de montaj,” — la fiecare modificare de cod aprobată, testează și publică automat noua versiune pe server.
OpenClaw	Componenta de integrare AI (asistentul conversațional din aplicație). Rulează izolat, fără drepturi de administrare.

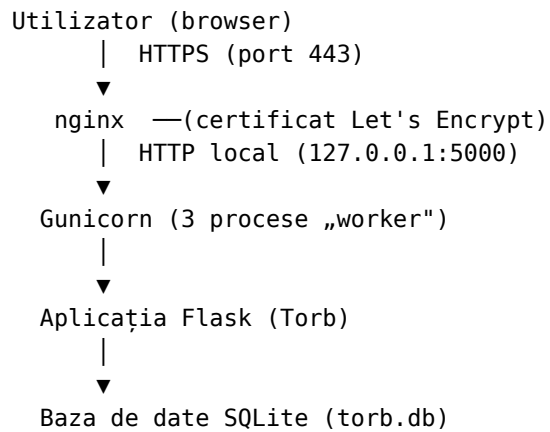
2.1. Două medii pe același server

Serverul găzduiește **două copii** ale aplicației, complet independente:

Mediu	Producție	Dezvoltare / Test
Adresă	<code>https://app.robrands.ro</code>	<code>https://app.robrands.ro:5001</code>
Scop	Folosit zilnic de utilizatori	Verificarea noilor versiuni înainte de producție
Bază de date	Reală	Copie separată, independentă

De ce două medii? Orice modificare nouă ajunge mai întâi pe mediul de **dezvoltare**, unde este testată automat. Doar după o **aprobare manuală** este publicată pe **producție**. Astfel, utilizatorii reali nu sunt afectați de versiuni nefinalizate.

2.2. Schema fluxului unei cereri



3. Serverul (VPS)

3.1. Date de identificare și acces

Proprietate	Valoare
Tip	Server virtual privat (VPS), Linux
Furnizor de hosting	cyberfolks.ro — date de logare în seiful de secrete (cap. 9)
Adresă IP publică	188.241.116.172
Domeniu	app.robrands.ro
Port SSH	2112 (neimplicit — nu 22)
Utilizator SSH	openclaw
Procesoare (vCPU)	2
Memorie RAM	2,8 GB
Disc	48 GB (/dev/sda1, montat pe /) — 16% folosit la 15.06.2026
Swap	0 (fără swap — vezi recomandarea din capitolul 11)
Backup la nivel de mașină	Realizat de furnizorul de hosting (mașina completă)

3.2. Conectarea prin SSH

Conectarea la server, de pe un calculator de administrare, se face cu:

```
ssh -p 2112 openclaw@188.241.116.172
# sau, după configurarea DNS:
ssh -p 2112 openclaw@app.robrands.ro
```

Începând cu 15.06.2026, autentificarea SSH se face **exclusiv cu cheie** — autentificarea cu parolă este **dezactivată** (PasswordAuthentication no). Conectarea necesită cheia privată personală pe calculatorul de administrare; cheia publică este în ~/.ssh/authorized_keys pe server. Parola contului openclaw rămâne validă doar pentru **consola de urgență** a furnizorului (cap. 9). Procedura completă: secțiunea „Configurarea și folosirea cheilor SSH”, din capitolul de securitate.

3.3. Panoul furnizorului (consola de urgență)

În situații în care SSH nu mai este disponibil (de exemplu blocarea accidentală a contului de administrare — vezi capitolul 11), accesul de urgență se face prin **consola web a furnizorului de hosting**, care oferă acces direct la mașină ca și cum ați sta în fața ei.

Furnizorul de hosting este **cyberfolks.ro**. Panoul de control, consola de urgență (VNC) și autentificarea în doi pași (Google Authenticator) — datele de logare se află în fișierul de secrete (capitolul 9), nu în acest manual.

4. Sistemul de operare și pachetele instalate

4.1. Sistemul de operare

Proprietate	Valoare
Distribuție	Ubuntu 24.04.4 LTS (Noble Numbat)
Kernel	6.8.0-117-generic, arhitectură x86_64

4.2. Pachete de sistem necesare

Pentru a reconstrui serverul este nevoie de următoarele pachete (denumirile sunt cele de pe distribuții Debian/Ubuntu; ajustați dacă distribuția diferă):

Pachet	Rol
nginx	Server web / reverse proxy
python3 (3.12) + python3-venv	Mediul de rulare al aplicației
git	Preluarea codului din GitHub
certbot + python3-certbot-nginx	Certificat HTTPS și reînnoire automată
ufw	Firewall
fail2ban	Blocarea atacurilor pe SSH
unattended-upgrades	Actualizări automate de securitate
sqlite3	Inspecția bazei de date (opțional, util)
nodejs + npm	Necesare pentru OpenClaw (integrarea AI)

4.3. Actualizări automate de securitate

unattended-upgrades este **activ** — serverul instalează singur actualizările de securitate ale sistemului de operare.

4.4. Versiunile exacte instalate

Verificate la 15.06.2026:

Componentă	Versiune
Python (sistem și venv)	3.12.3
Gunicorn	26.0.0
nginx	1.24.0 (Ubuntu)
Node.js	v22.22.2
npm	10.9.7
OpenClaw	2026.6.6

Componentă	Versiune
Certbot	2.9.0

5. Utilizatori, permisiuni și proprietatea fișierelor

Înțelegerea utilizatorilor de pe server este esențială pentru securitate și pentru reproducere.

5.1. Utilizatorii relevanți

Utilizator	Rol și apartenențe
openclaw	Cont de administrare și implementare (SSH + GitHub Actions). uid 999, grupuri: openclaw, www-data. Trebuie să fie și în grupul sudo (vezi atenționarea de mai jos). Repornirea serviciilor la implementare funcționează printr-o regulă sudoers dedicată (torb-deploy), nu prin apartenența la grup.
www-data	Cont de serviciu — sub el rulează Unicorn (aplicația). uid 33, grup: www-data. Nu are drepturi de administrare.

LECȚIE IMPORTANTĂ (13.06.2026): openclaw este **singurul** cont de administrare și nu există o parolă de root separată. Scoaterea lui din grupul sudo a blocat accesul de administrare. **Păstrați mereu openclaw în grupul sudo.** Recuperarea, dacă se întâmplă, se face din consola furnizorului cu `usermod -aG sudo openclaw`. Gaura reală de securitate fusese grupul docker (echivalent root, fără parolă), nu sudo (protejat cu parolă).

5.2. Proprietatea și permisiunile fișierelor

Cale	Proprietar / mod
/var/www/html/torb-py	www-data:www-data, mod 775 (scriere de grup)
data/ și torb.db	www-data:www-data, mod 775
data/backups/	www-data:www-data, mod 2775 (setgid — fișierele create de openclaw moștenesc grupul www-data)
.env (prod și dev)	openclaw:www-data, mod 640 (secretele nu sunt lizibile pentru „ceilalți”)

De ce poate openclaw să scrie în arborele aplicației deși aplicația rulează ca www-data? Pentru că openclaw este în grupul www-data, iar directoarele sunt scriibile de grup. Astfel implementarea (care rulează ca openclaw) poate actualiza fișierele aplicației.

6. Aplicația: medii, servicii și fișiere

6.1. Producție

Proprietate	Valoare
URL	https://app.robrands.ro
Cale pe disc	/var/www/html/torb-py
Serviciu systemd	torb-py
Adresă Unicorn	127.0.0.1:5000 (numai local)
Procese Unicorn	3
Rulează ca	www-data
Mediu virtual Python	/var/www/html/torb-py/venv
Bază de date	/var/www/html/torb-py/data/torb.db
Fișier de mediu	/var/www/html/torb-py/.env
Jurnale	/var/www/html/torb-py/logs/app.log
Declanșator implementare	Push pe ramura main (GitHub Actions)

6.2. Dezvoltare / Test

Proprietate	Valoare
URL	https://app.robrands.ro:5001
Cale pe disc	/var/www/html/torb-py-dev
Serviciu systemd	torb-dev
Adresă Unicorn	127.0.0.1:5002 (numai local)
Procese Unicorn	2
Rulează ca	www-data
Mediu virtual Python	/var/www/html/torb-py-dev/venv
Bază de date	/var/www/html/torb-py-dev/data/torb.db (copie independentă)
Fișier de mediu	/var/www/html/torb-py-dev/.env
Jurnale	/var/www/html/torb-py-dev/logs/app.log

6.3. Punctul de intrare al aplicației (WSGI)

Obiectul aplicației este creat de fabrica `create_app()` și expus ca `app` în `app/app.py` (`app = create_app()`). Unicorn este pornit cu ținta `app:app:app` (modulul `app.app`, obiectul `app`).

Există și un `wsgi.py` în rădăcina proiectului (`app = create_app()`, deci ținta `wsgi:app`), echivalent funcțional. Unitatea `systemd` reală folosește însă `app:app:app` — vezi mai jos.

6.4. Serviciile systemd

Aplicația este gestionată ca **serviciu de sistem**, deci pornește automat la repornirea serverului.

```
sudo systemctl status torb-py      # producție
sudo systemctl status torb-dev     # dezvoltare
sudo systemctl restart torb-py
sudo systemctl restart torb-dev
```

Fișierele unităților:

- /etc/systemd/system/torb-py.service
- /etc/systemd/system/torb-dev.service

Conținutul real al unității de **producție** (/etc/systemd/system/torb-py.service):

```
[Unit]
Description=Tor Logistic Flask App
After=network.target

[Service]
User=www-data
WorkingDirectory=/var/www/html/torb-py
EnvironmentFile=/var/www/html/torb-py/.env
ExecStart=/var/www/html/torb-py/venv/bin/gunicorn --workers 3 --bind 127.0.0.1:5000
app.app:app
Restart=always
StandardOutput=append:/var/www/html/torb-py/logs/app.log
StandardError=append:/var/www/html/torb-py/logs/app.log

[Install]
WantedBy=multi-user.target
```

Unitatea de **dezvoltare** (/etc/systemd/system/torb-dev.service) este identică, cu: cale / var/www/html/torb-py-dev, --workers 2, --bind 127.0.0.1:5002 și jurnalele în directorul dev.

Restricție de securitate: serviciul torb-py rulează **fără** NoNewPrivileges. Acest lucru este **obligatoriu** pentru ca integrarea OpenClaw (sudo -u openclaw) să funcționeze. Dacă o viitoare întărire a securității adaugă NoNewPrivileges=true, integrarea AI se strică — eliminați linia, rulați systemctl daemon-reload și reporniți serviciul.

7. nginx, HTTPS și firewall

7.1. nginx (reverse proxy)

nginx primește traficul din internet și îl trimite către Unicorn. Fișiere de configurare:

Fișier	Rol
/etc/nginx/sites-available/app.robrands.ro	Site producție (gestionat de Certbot)
/etc/nginx/sites-available/torb-dev	Site dezvoltare (port 5001)
/etc/nginx/sites-enabled/default	Rezervă — redirecționează către 127.0.0.1:5000

Rezumatul rutării:

```
:80          → redirecționare 301 către https://app.robrands.ro
:443 ssl     → nginx → 127.0.0.1:5000  (Unicorn producție)
:5001 ssl    → nginx → 127.0.0.1:5002  (Unicorn dezvoltare)
```

Conținutul real al vhost-ului de **producție** (/etc/nginx/sites-available/app.robrands.ro, gestionat de Certbot):

```
server {
    server_name app.robrands.ro www.app.robrands.ro;
    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
    listen 443 ssl; # managed by Certbot
    ssl_certificate /etc/letsencrypt/live/app.robrands.ro/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/app.robrands.ro/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
}
server { # redirecționare HTTP → HTTPS (managed by Certbot)
    if ($host = www.app.robrands.ro) { return 301 https://$host$request_uri; }
    if ($host = app.robrands.ro)     { return 301 https://$host$request_uri; }
    listen 80;
    server_name app.robrands.ro www.app.robrands.ro;
    return 404;
}
```

Conținutul real al vhost-ului de **dezvoltare** (/etc/nginx/sites-available/torb-dev):

```
server { # acces prin IP pe :80 → HTTPS
    listen 80;
    server_name 188.241.116.172;
    return 301 https://app.robrands.ro$request_uri;
}
```

```
server {
    listen 5001 ssl;
    server_name app.robrands.ro;
    ssl_certificate /etc/letsencrypt/live/app.robrands.ro/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/app.robrands.ro/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;
    location / {
        proxy_pass http://127.0.0.1:5002;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Anteturile X-Forwarded-For / X-Forwarded-Proto setate de nginx sunt esențiale: aplicația folosește ProxyFix ca să vadă IP-ul real al vizitatorului (pentru limitarea ratei la login și pentru jurnalul de audit).

Comenzi utile:

```
sudo nginx -t          # verifică sintaxa configurației
sudo systemctl reload nginx # reîncarcă fără întreruperea serviciului
```

7.2. HTTPS (Let's Encrypt / Certbot)

Certificatul SSL este gratuit, emis de Let's Encrypt prin Certbot, și se **reînnoiește automat**.

Proprietate	Valoare
Director certificat	/etc/letsencrypt/live/app.robrands.ro/
Fișiere	fullchain.pem, privkey.pem (plus cert.pem, chain.pem)
Domenii acoperite	app.robrands.ro, www.app.robrands.ro
Tip cheie	ECDSA
Expiră	2026-08-23 (valid 69 zile la 15.06.2026)
Reînnoire	automată, cu 30 de zile înainte de expirare (renew_before_expiry = 30 days)
Autenticator / installer	nginx
Partajare	Același certificat este folosit de ambele blocuri nginx (prod + dev)

Reînnoirea automată se face prin **timer systemd** certbot.timer (Certbot 2.9.0), care rulează de două ori pe zi și reînnoiește certificatul când se apropie de expirare. Verificare:

```
systemctl list-timers '*certbot*' # afișează următoarea rulare
sudo certbot certificates          # afișează data de expirare (necesită sudo)
```

7.3. Firewall (ufw)

Firewall-ul este **activ** (politică implicită: deny la intrare, allow la ieșire). Reguli efective (verificat 15.06.2026, IPv4 + IPv6):

Port	Protocol	Scop
2112	TCP	SSH (singurul port pe care ascultă serviciul)
80	TCP	HTTP (redirecționează către HTTPS)
443	TCP	HTTPS — producție
5001	TCP	HTTPS — dezvoltare / test

Remediat 15.06.2026: regulile vechi 22/tcp și 8888/tcp (SSH fără serviciu în spate) și regula Tailscale 41641/udp au fost eliminate. Firewall-ul expune acum doar porturile de mai sus.

7.4. fail2ban

fail2ban este **activ**, cu jail-ul [sshd] pornit — blochează automat IP-urile care încearcă parole greșite repetat. Stare la 15.06.2026: 126 încercări eșuate cumulate, 0 IP-uri blocate în prezent. Configurarea (maxretry / bantime) se află în /etc/fail2ban/jail.local sau jail.d/.

7.5. Configurarea SSH (sshd) și întărirea recomandată

Valori efective (sshd -T, 15.06.2026):

Parametru	Valoare curentă
Port	2112
PermitRootLogin	prohibit-password (întărit 15.06.2026)
PubkeyAuthentication	yes
PasswordAuthentication	no (dezactivat 15.06.2026 — doar chei)

Întărirea se aplică prin fișierul /etc/ssh/sshd_config.d/99-torb-hardening.conf, urmat de `sudo systemctl reload ssh`.

Finalizat 15.06.2026: a fost configurată o cheie SSH personală pentru admin și **autentificarea cu parolă a fost dezactivată** (PasswordAuthentication no). Logarea ca root rămâne prohibit-password. SSH acceptă acum exclusiv chei. Recuperare în caz de pierdere a cheii: consola cyberfolks (login cu parola contului — funcționează pentru că nu este SSH).

7.6. Configurarea și folosirea cheilor SSH

Autentificarea SSH se face **exclusiv cu cheie** (parola este dezactivată). O cheie SSH are două părți: o **cheie privată** (rămâne pe calculatorul de administrare, secretă) și o **cheie publică** (se instalează pe server, în ~/.ssh/authorized_keys). La conectare, serverul verifică criptografic că dețineți cheia privată — nu se transmite nicio parolă.

7.6.1. Bootstrap: cum ajunge prima cheie pe server

Cheia publică **trebuie** scrisă în ~/.ssh/authorized_keys de pe server, deci este nevoie de un acces inițial. Trei căi posibile:

1. **Un login SSH existent** — cu parolă (cât timp era activă) sau cu o cheie deja funcțională.
2. **Consola furnizorului (cyberfolks VNC)** — login local cu parola contului, apoi editarea fișierului. Funcționează chiar dacă SSH este complet nefuncțional.
3. **La crearea serverului** — multe servicii cloud permit injectarea unei chei publice la provisioning.

Cheia publică **nu este secretă**: un coleg nou vă poate trimite fișierul `.pub`, iar dvs. îl adăugați în `authorized_keys` — fără a avea nevoie de parola sau de cheia privată a celui alt.

7.6.2. Generarea unei chei (pe calculatorul de administrare — Windows PowerShell)

```
ssh-keygen -t ed25519 -C "vlad-laptop" -f $env:USERPROFILE\.ssh\id_ed25519
```

Creează `id_ed25519` (privată — nu se distribuie niciodată) și `id_ed25519.pub` (publică). Setati o **passphrase** (protejează cheia dacă vă este furat calculatorul).

7.6.3. Instalarea cheii publice pe server

Cât timp aveți încă acces (parolă sau altă cheie), dintr-o singură comandă:

```
Get-Content $env:USERPROFILE\.ssh\id_ed25519.pub | ssh -p 2112  
openclaw@188.241.116.172 "cat >> ~/.ssh/authorized_keys && chmod 600 ~/.ssh/  
authorized_keys"
```

7.6.4. Conectarea

```
ssh -p 2112 openclaw@188.241.116.172
```

Opțional, definiți un alias în `~/.ssh/config` ca să tastați doar `ssh torb`:

```
Host torb  
  HostName 188.241.116.172  
  Port 2112  
  User openclaw  
  IdentityFile ~/.ssh/id_ed25519
```

7.6.5. Mai multe calculatoare

`authorized_keys` poate conține mai multe chei. Generați câte o pereche **pe fiecare calculator** și adăugați fiecare `.pub` ca linie separată. Astfel puteți **revoaca individual** un dispozitiv (ștergând linia respectivă) fără a afecta celelalte. **Nu** copiați aceeași cheie privată pe mai multe mașini.

Nu dezactivați parola SSH (`PasswordAuthentication no`) decât **după** ce ați confirmat logarea cu cheie într-un al doilea terminal. În caz de blocare, recuperarea se face din consola cyberfolks (login cu parola contului — funcționează pentru că nu este SSH), de unde reinstalați o cheie publică.

7.7. Tailscale (rețea privată mesh)

Tailscale (VPN mesh) era instalat dar deconectat („Logged out,“). La 15.06.2026 a fost **dezactivat** (`sudo systemctl disable --now tailscaled`) și portul 41641/udp a fost închis în firewall, nefiind utilizat.

Dacă pe viitor este nevoie de acces privat mesh: `sudo systemctl enable --now tailscaled`, `sudo tailscale up`, redeschideți portul (`sudo ufw allow 41641/udp`) și documentați tailnet-ul + identitatea în seiful de secrete (cap. 9).

8. Variabile de mediu și informații sensibile

8.1. Fișierul .env

Fiecare mediu are propriul fișier .env (mod 640, `openclaw:www-data`). El conține atât **secrete**, cât și **configurări ne-sensibile**. Secretele sunt injectate automat de GitHub Actions la fiecare implementare (vezi capitolul 8); valorile ne-sensibile sunt scrise direct de scriptul de implementare.

8.2. Inventarul valorilor (FĂRĂ valori aici)

Acest manual NU conține valori secrete. Tabelul de mai jos enumeră **ce** secrete există și **unde** sunt folosite. Valorile reale se păstrează separat — vezi secțiunea 9.4.

Nume secret	Folosit pentru
FLASK_SECRET_KEY	Semnarea sesiunilor / cookie-urilor Flask
ANTHROPIC_API_KEY	Apelurile către modelul AI (Claude) din aplicație
EMAG_USERNAME / EMAG_PASSWORD / EMAG_WAREHOUSE_ID	Sincronizarea stocurilor pe eMAG Marketplace
SHOPIFY_CLIENT_ID / SHOPIFY_CLIENT_SECRET	Autentificarea OAuth pentru Shopify
VPS_HOST / VPS_USERNAME / VPS_SSH_KEY	Acces SSH al GitHub Actions la server (cheia de implementare)

Valori **ne-sensibile** scrise direct de scriptul de implementare (nu sunt secrete):

SHOPIFY_SHOP_DOMAIN=basilur-tea-romania.myshopify.com
SHOPIFY_LOCATION_ID=110603567429
SHOPIFY_STOCK_SAFETY_THRESHOLD=5
EMAG_STOCK_SAFETY_THRESHOLD=5
SESSION_COOKIE_SECURE=1

8.3. Email / SMTP (resetare parolă)

Funcția de **resetare a parolei** trimite emailuri prin SMTP (modulul Python `smtplib`, cu `STARTTLS`). Cod: `app/blueprints/auth.py`. Configurarea se face **exclusiv prin variabile de mediu** în .env:

Variabilă	Rol
SMTP_HOST	Serverul SMTP. Dacă lipsește, trimiterea e dezactivată , iar resetarea „degradează”, elegant (pagina anunță că resetarea prin email nu este configurată).
SMTP_PORT	Implicit 587 (STARTTLS)
SMTP_USER SMTP_PASSWORD	/ Autentificare; login-ul este omis dacă sunt goale
SMTP_FROM	Adresa expeditor (implicit = SMTP_USER)

Configurare curentă: **Gmail** (smtp.gmail.com:587, STARTTLS) cu un **App Password** Gmail (nu parola contului Google); valorile exacte se află în seiful de secrete (secțiunea SMTP).

Actualizat 15.06.2026: SMTP_USER și SMTP_PASSWORD sunt acum **injectate de GitHub Actions** din secrete (ca celelalte secrete, în ambele job-uri de deploy), iar SMTP_HOST/SMTP_PORT/SMTP_FROM (ne-sensibile) se află în .env.example. Scriptul de deploy nu rescrie aceste trei din urmă, deci trebuie să rămână prezente în .env-ul serverului (sunt). **Atenție:** secretele SMTP_USER și SMTP_PASSWORD trebuie să existe în GitHub, altfel deploy-ul scrie valori goale și emailul de resetare nu mai funcționează.

8.4. Unde sunt stocate valorile reale

Valorile secrete trăiesc în **trei** locuri, fiecare cu rol propriu:

Locație	Rol
GitHub Actions Secrets	Sursa de adevăr pentru implementare. Se setează în GitHub: <i>Settings</i> → <i>Secrets and variables</i> → <i>Actions</i> . De aici sunt injectate automat în .env la fiecare implementare.
.env de pe server	Copia de lucru, generată automat. Nu se editează manual.
Seif de secrete (KeePassXC)	Copie de referință pentru oameni (recuperare, audit, predare). Vezi mai jos.

8.5. Seiful de secrete (KeePassXC)

Valorile sensibile se păstrează într-un **seif de parole criptat** (KeePassXC), ținut **în afara git**, nu într-un fișier text simplu (migrare 2026-06-17):

Proprietate	Valoare
Fișier	torb-secrets.kdbx (format KeePassXC)
Locație	pe calculatorul de administrare, în afara depozitului (de ex. C:\Users\<user>\Vault\)
Criptare	AES-256 + derivare de cheie Argon2id; deblocat cu o frază de acces principală
Versionat în git?	Nu în depozitul aplicației (ar ajunge pe serverul de producție la deploy). Fiind criptat, poate fi păstrat pentru backup/

Proprietate	Valoare
	partajare într-un depozit privat separat sau un folder sincronizat în cloud.

Reguli obligatorii pentru seiful de secrete:

- Nu adăugați **niciodată** `.kdbx` în depozitul aplicației (TorbApp) — ar fi livrat pe server la deploy. `.gitignore` blochează `*.kdbx`.
- Păstrați **fraza de acces principală** într-un al doilea loc de încredere (manager de parole personal sau notată într-un loc fizic sigur). Dacă o uitați și nu aveți backup, conținutul este **irecuperabil**.
- Păstrați o copie de siguranță a fișierului `.kdbx` **în afara calculatorului** (folder sincronizat în cloud sau depozit privat dedicat — e criptat, deci sincronizarea e sigură).
- La plecarea unei persoane cu acces, **roțiți** secretele afectate și schimbați fraza de acces.

De ce un seif și nu un fișier text: `.kdbx` este criptat pe disc, deci poate fi păstrat ca backup și partajat fără a expune valorile. Cele mai multe secrete sunt oricum duplicate în GitHub Actions Secrets și în `.env`-ul serverului; singura valoare cu adevărat unică este parola consolei de urgență a furnizorului de hosting.

9. Implementare automată (CI/CD — GitHub Actions)

9.1. Depozitul de cod

GitHub: <https://github.com/ifrimdrag-hue/TorbApp>. Ramura de producție: main.

Directoare „sigure”, pentru git, configurate pe server:

```
git config --global --add safe.directory /var/www/html/torb-py
git config --global --add safe.directory /var/www/html/torb-py-dev
```

Cheia de deploy: GitHub Actions se conectează prin SSH (port 2112) cu cheia VPS_SSH_KEY, instalată în ~/.ssh/authorized_keys al contului openclaw (comentariu github-deploy, amprentă SHA256:EigIB4...). La 15.06.2026 s-au eliminat din authorized_keys o linie-reziduu (paste-public-key-here) și o cheie de deploy veche, rotită (github-actions-deploy, nefolosită din 25.05.2026). Copii de siguranță: ~/.ssh/authorized_keys.bak.*.

9.2. Conducă de implementare

Fișier: .github/workflows/deploy_VPS.yml. La fiecare push pe main, rulează în ordine:

1. **lint** — verificare stil cod (ruff check .)
2. **test** — testele automate (pytest)
3. **security** — scanare vulnerabilități dependențe (pip-audit) — **blocant**
4. **deploy_dev** — implementare pe mediul de dezvoltare
5. **test_dev** — teste „de fum”, pe <https://app.robrands.ro:5001>
6. **approve_prd** — **poartă de aprobare manuală** (GitHub Environment production)
7. **deploy_prd** — implementare pe producție
8. **test_prd** — teste „de fum”, pe <https://app.robrands.ro>

Pentru cititori non-tehnici: nicio modificare nu ajunge automat la utilizatori. După ce trece testele și este publicată pe mediul de dezvoltare, **o persoană trebuie să apese „Approve,”** în GitHub pentru ca versiunea să meargă în producție.

9.3. Cum funcționează o implementare

- Conectare prin SSH (acțiunea appleboy/ssh-action, port 2112).
- Pe server: git fetch + git reset --hard origin/main (sursa de adevăr este git; modificările locale accidentale se pierd).
- Pe producție se șterg fișierele non-runtime (tests/, context/, docs/).
- Secretele se injectează în .env, apoi se aplică permisiunile (chgrp www-data .env && chmod 640 .env).
- Se actualizează dependențele (pip install -r requirements.txt).
- **Producție:** se face un backup pre-deploy al bazei de date **înainte** de migrări.
- Se rulează migrările bazei de date (python migrations/runner.py data/torb.db) — o migrare eșuată **anulează** implementarea și lasă serviciul vechi neatins.
- Se repornește serviciul (sudo systemctl restart torb-py / torb-dev).

9.4. Revenirea la versiunea anterioară (rollback)

La implementarea de producție se salvează SHA-ul anterior în `.previous_deploy_sha`. Există un workflow manual „**Rollback Production**„ (tab-ul Actions din GitHub) care readuce versiunea precedentă; necesită tastarea `ROLLBACK` pentru confirmare.

Notă tehnică: repornirea serviciilor la implementare nu folosește apartenența la grupul `sudo`, ci o regulă `sudoers` dedicată (`torb-deploy`) care permite lui `openclaw` să ruleze **exact** `systemctl restart torb-py` și `systemctl restart torb-dev`, fără parolă. De aceea implementările funcționează chiar și cât timp `openclaw` nu este în grupul `sudo`.

9.5. Dezvoltare locală (Windows)

`unicorn` se instalează din `requirements.txt` doar pe Linux (marcator de mediu). Pe Windows, dezvoltarea locală folosește `waitress`. Lansarea locală: `Start-Hub.bat` / `tools\Start-Hub.ps1`.

10. Procedura de reconstrucție de la zero (runbook)

Această secțiune permite **reconstruirea completă** a infrastructurii pe un server nou. Presupune un VPS proaspăt cu Debian/Ubuntu și acces de administrare.

Multe comenzi necesită `sudo`. Pe serverul curent, contul `openclaw` trebuie să fie în grupul `sudo` (vezi capitolul 11). Pe un server nou, executați ca utilizator cu drepturi de administrare.

10.1. Pasul 1 — Sistem de bază și pachete

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y nginx python3 python3-venv git \
    certbot python3-certbot-nginx ufw fail2ban \
    unattended-upgrades sqlite3 nodejs npm
sudo dpkg-reconfigure --priority=low unattended-upgrades # activează update-urile automate
```

10.2. Pasul 2 — Utilizatori și grupuri

```
sudo adduser openclaw
sudo usermod -aG sudo,www-data openclaw # PĂSTRAȚI openclaw în sudo (vezi cap. 11)
# instalați cheia publică SSH în /home/openclaw/.ssh/authorized_keys
```

10.3. Pasul 3 — SSH (port 2112) și firewall

```
# SSH pe portul 2112
echo 'Port 2112' | sudo tee /etc/ssh/sshd_config.d/10-port.conf
sudo sshd -t && sudo systemctl reload ssh

# Firewall
sudo ufw allow 2112/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw allow 5001/tcp
sudo ufw enable
```

10.4. Pasul 4 — Codul aplicației (prod + dev)

```
sudo mkdir -p /var/www/html/torb-py /var/www/html/torb-py-dev
sudo chown www-data:www-data /var/www/html/torb-py /var/www/html/torb-py-dev
sudo chmod 775 /var/www/html/torb-py /var/www/html/torb-py-dev

# ca utilizator cu acces la grupul www-data:
cd /var/www/html/torb-py
git clone https://github.com/ifrimdrag-hue/TorbApp.git .
git config --global --add safe.directory /var/www/html/torb-py
python3 -m venv venv
venv/bin/pip install -r requirements.txt
# repetați identic pentru /var/www/html/torb-py-dev
```

10.5. Pasul 5 — Fișierul .env

Creați .env în fiecare mediu, completând secretele din seiful de secrete (cap. 9) și valorile ne-sensibile din secțiunea 9.3. Apoi:

```
sudo chgrp www-data .env && sudo chmod 640 .env
```

10.6. Pasul 6 — Baza de date și directoarele de backup

```
mkdir -p data/backups logs
sudo chown -R www-data:www-data data logs
sudo chmod 2775 data/backups # setgid
python migrations/runner.py data/torb.db # creează / actualizează schema
```

10.7. Pasul 7 — Serviciile systemd

Creați /etc/systemd/system/torb-py.service și torb-dev.service (vezi modelele din secțiunea 6.4), apoi:

```
sudo systemctl daemon-reload
sudo systemctl enable --now torb-py torb-dev
```

10.8. Pasul 8 — nginx și HTTPS

```
# creați vhost-urile (vezi cap. 8.1 / Anexa A), activați-le:
sudo ln -s /etc/nginx/sites-available/app.robrands.ro /etc/nginx/sites-enabled/
sudo nginx -t && sudo systemctl reload nginx
```

```
# certificat HTTPS (DNS-ul domeniului trebuie să indice deja spre IP)
sudo certbot --nginx -d app.robrands.ro
```

10.9. Pasul 9 — Backup zilnic (cron)

```
sudo -u www-data crontab -e
# adăugați linia:
30 2 * * * cd /var/www/html/torb-py && venv/bin/python etl/backup_db.py backup --tag
daily >> logs/backup.log 2>&1
```

10.10. Pasul 10 — OpenClaw (opțional, integrarea AI)

Vezi capitolul 12 pentru instalarea și configurarea completă a gateway-ului OpenClaw.

10.11. Pasul 11 — Verificare finală

```
curl -s -o /dev/null -w "%{http_code}\n" https://app.robrands.ro/healthz # 200
curl -s -o /dev/null -w "%{http_code}\n" https://app.robrands.ro:5001/healthz # 200
```

11. Stare curentă și acțiuni în așteptare

Stare 15.06.2026 — REZOLVAT. Contul `openclaw` a fost repus în grupul `sudo` (tichet rezolvat la furnizor). Lista de verificare de mai jos a fost **parcursă**: firewall curățat (22/8888/41641 eliminate), logare root restricționată (`prohibit-password`), Tailscale dezactivat, gateway OpenClaw reciclat, `.env` confirmat 640, `server_tokens` off aplicat în `nginx`, iar **autentificarea SSH cu parolă a fost dezactivată** (`PasswordAuthentication no`) după configurarea unei chei personale de admin. Toate punctele listei de mai jos sunt finalizate. Istoricul blocajului din 13.06.2026 se păstrează mai jos ca referință.

Context istoric (13.06.2026): contul `openclaw` fusese scos din grupul `sudo` și era **singurul** cont de administrare, fără parolă root separată — administratorul a rămas blocat de la accesul de administrare până la intervenția din consola furnizorului. Lecția: **păstrați mereu `openclaw` în grupul `sudo`.**

11.1. Lista de verificare după restabilirea accesului `sudo`

De rulat **în ordine**, imediat după ce `openclaw` revine în grupul `sudo`:

```
# 1. Confirmare recuperare
id openclaw          # se așteaptă: ...,27(sudo),33(www-data)
sudo -v              # trebuie să reușească

# 2. Întărire SSH — ☒ APLICAT 15.06.2026 (root prohibit-password + parolă
dezactivată)
sudo tee /etc/ssh/sshd_config.d/99-torb-hardening.conf >/dev/null <<'EOF'
PermitRootLogin prohibit-password
PasswordAuthentication no    # activat DOAR după confirmarea logării cu cheie (vezi
secțiunea chei SSH)
EOF
sudo sshd -t && sudo systemctl reload ssh

# 3. Ascunderea versiunii nginx: adăugați `server_tokens off;` în http{} din
# /etc/nginx/nginx.conf, apoi:
sudo nginx -t && sudo systemctl reload nginx

# 4. Reciclarea sesiunii OpenClaw (elimină grupurile vechi docker/sudo din procesul
viu)
sudo loginctl terminate-user openclaw    # linger repornește gateway-ul
sleep 5                                  # apoi verificați că gateway-ul e sus

# 5. Firewall — elimină regulile SSH vechi (serviciul ascultă doar pe 2112)
sudo ufw status verbose
sudo ufw delete allow 22/tcp && sudo ufw delete allow 8888/tcp
sudo fail2ban-client status sshd

# 6. Tailscale — instalat dar deconectat + 41641/udp deschis. Alegeți UNA:
# (a) dacă NU e necesar:
sudo systemctl disable --now tailscaled && sudo ufw delete allow 41641/udp
```



```
# (b) dacă e necesar: sudo tailscale up, apoi documentați tailnet-ul în seiful de secrete
```

```
# 7. Curățenie (opțional)
```

```
ls -l /var/www/html/torb-py/.env
```

```
# 640 openclaw:www-data
```

```
apt-get -s upgrade | grep -i security
```

```
# update-uri de securitate în așteptare
```

11.2. Recomandare pe termen lung

Rularea gateway-ului OpenClaw sub un **cont dedicat, cu privilegii minime**, separat de contul de administrare/implementare openclaw — elimină cauza comună (identitate partajată) atât a riscului de execuție, cât și a blocajului de mai sus.

11.3. De adăugat: swap

Serverul are **0 swap** (2 vCPU / 2,8 GB RAM). Recomandare: adăugați un fișier swap pentru reziliență la vârfuri de memorie:

```
sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile
```

```
sudo mkswap /swapfile && sudo swapon /swapfile
```

```
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

12. Baza de date și backup-uri

12.1. Baza de date

O singură bază de date SQLite (data/torb.db) per mediu. Conexiunile folosesc WAL și busy_timeout (5s) pentru rezistență la accesul simultan.

12.2. Sistemul de backup (numai producție)

Proprietate	Valoare
Locație	/var/www/html/torb-py/data/backups/
Format	torb_AAAA-LL-ZZ_HHMMSS_<etichetă>.db.gz
Etichete	daily, pre-deploy, pre-restore, manual
Păstrare	15 zile, păstrând mereu cele mai recente 3
Backup zilnic	cron (www-data), 02:30
Backup pre-implementare	Job-ul CI deploy_prd, înainte de migrări
Interfață admin	https://app.robrands.ro/admin/db — listă, backup manual, descărcare, restaurare protejată
Motor	API-ul de backup online SQLite (sigur pe o bază de date activă)

Comenzi (din /var/www/html/torb-py, cu mediul virtual activat):

```
python etl/backup_db.py list
python etl/backup_db.py backup --tag manual
python etl/backup_db.py restore torb_2026-06-11_023000_daily.db.gz
```

Restaurarea creează automat un backup pre-restore, re-rulează migrările (backup-urile vechi sunt aduse la schema curentă) și necesită repornirea serviciului după (sudo systemctl restart torb-py) pentru ca toate procesele Gunicorn să își golească memoria cache. Mediul de dezvoltare nu are backup-uri programate, prin design. Detalii pentru utilizatori: vezi **Manualul de Mentenanță Bază de Date**.

12.3. Niveluri de protecție a datelor

1. **Backup-ul VPS al furnizorului** — întreaga mașină (recuperare în caz de dezastru hardware).
2. **Backup-urile bazei de date** — granulare, frecvente (recuperare din erori de date / implementări greșite).
3. **Descărcare manuală** din /admin/db — pentru o copie în afara serverului.

13. OpenClaw (integrarea AI)

OpenClaw este componenta care alimentează asistentul conversațional AI din aplicație. Rulează **izolat și fără drepturi de execuție pe server** (chat-only, prin design).

13.1. Cum rulează

OpenClaw (Node.js) rulează ca serviciu systemd **de utilizator** (user-linger) sub contul openclaw — **nu** la nivel de sistem. Administrare:

```
export XDG_RUNTIME_DIR=/run/user/${id -u}      # rulat ca utilizatorul openclaw
systemctl --user status openclaw-gateway.service
systemctl --user restart openclaw-gateway.service
```

Proprietate	Valoare
Serviciu	openclaw-gateway.service (unitate de utilizator, ~/.config/systemd/user/)
Pornire	node ~/.local/lib/node_modules/openclaw/dist/index.js gateway --port 18789
Ascultă	127.0.0.1:18789 (gateway) + 127.0.0.1:18791 (control browser) — numai local
Configurare	~/.openclaw/openclaw.json (format JSON5)
Furnizor model	OpenRouter, model openrouter/free
Permisuni execuție	deny-all — agentul nu poate rula comenzi pe server

13.2. Integrarea cu Flask

Ruta /admin/openclaw-ask din app/app.py apelează (prin sudo -n -u openclaw) un script wrapper care comunică cu agentul. S-a ales calea prin linia de comandă (nu WebSocket) pentru că gateway-ul folosește un handshake greu de reimplementat, iar CLI-ul corespunde mereu versiunii instalate.

Componente de pe server:

```
# Wrapper: /usr/local/bin/torb-openclaw-ask (root, mod 0755)
#!/usr/bin/env bash
set -euo pipefail
export HOME=/home/openclaw
export XDG_RUNTIME_DIR="/run/user/${id -u}"
export PATH="/home/openclaw/.local/bin:/usr/bin:/bin"
exec openclaw agent --agent main --session-id "${1:?}" -m "${2:?}" --json

# Sudoers: /etc/sudoers.d/torb-openclaw (mod 0440)
www-data ALL=(openclaw) NOPASSWD: /usr/local/bin/torb-openclaw-ask
```

Fluxul complet `www-data → sudo -u openclaw → wrapper → agent → gateway → OpenRouter` a fost verificat funcțional. Cheia API a furnizorului este per-agent (`~/.openclaw/agents/main/agent/openclaw-agent.sqlite`); după schimbarea ei, **reporniți gateway-ul**.

14. Mentenanță de rutină și depanare

14.1. Comenzi utile

```
# Urmărirea jurnalelor
sudo tail -f /var/www/html/torb-py/logs/app.log
sudo tail -f /var/www/html/torb-py-dev/logs/app.log

# Test rapid de funcționare
curl -s -o /dev/null -w "%{http_code}\n" https://app.robrands.ro/healthz
curl -s -o /dev/null -w "%{http_code}\n" https://app.robrands.ro:5001/healthz

# Starea serviciilor
sudo systemctl status torb-py torb-dev nginx

# nginx
sudo nginx -t && sudo systemctl reload nginx
```

14.2. Probleme frecvente

Situație	Ce puteți face
Aplicația nu răspunde (eroare 502)	Verificați Unicorn: <code>sudo systemctl status torb-py</code> ; reporniți: <code>sudo systemctl restart torb-py</code> ; consultați <code>logs/app.log</code> .
Certificatul HTTPS a expirat	Reînnoiți: <code>sudo certbot renew</code> apoi <code>sudo systemctl reload nginx</code> . Verificați timerul de reînnoire automată.
Implementarea eșuează la migrări	Serviciul vechi rămâne pornit. Consultați jurnalul rulării în GitHub Actions; corectați migrarea și re-publicați.
„database is locked„	Rar (WAL + <code>busy_timeout</code> activ). Dacă persistă, semn că e momentul migrării la Postgres (vezi backlog tehnic).
Spațiu pe disc insuficient (backup eșuat)	<code>df -h</code> ; ștergeți backup-uri vechi descărcate deja; verificați <code>data/backups/</code> .
Nu mă pot conecta prin SSH	Confirmați portul 2112 și cheia privată. Dacă tot eșuează, folosiți consola furnizorului (cap. 3.3).
Asistentul AI nu răspunde	Reciclați sesiunea: <code>ca openclaw, systemctl --user restart openclaw-gateway.service</code> ; verificați cheia API per-agent.

15. Anexa A — Comenzi pentru completarea informațiilor lipsă

Rulați aceste comenzi pe server și completați secțiunile marcate cu „X DE COMPLETAT”.

15.1. Bloc A — fără sudo (rulabil acum, ca openclaw)

```
cat /etc/os-release; uname -a
nproc; free -h; df -h /; swapon --show
python3 --version
/var/www/html/torb-py/venv/bin/python --version
/var/www/html/torb-py/venv/bin/gunicorn --version
node --version; npm --version; openclaw --version 2>/dev/null || ~/.local/bin/
openclaw --version
nginx -v
cat /etc/systemd/system/torb-py.service
cat /etc/systemd/system/torb-dev.service
cat /etc/nginx/sites-available/app.robrands.ro
cat /etc/nginx/sites-available/torb-dev
systemctl list-timers '*certbot*' --all; certbot --version 2>/dev/null
getent hosts app.robrands.ro
```

15.2. Bloc B — necesită sudo (rulabil după revenirea openclaw în grupul sudo)

```
sudo ufw status verbose
sudo fail2ban-client status sshd
sudo sshd -T | grep -Ei 'port|permitroot|passwordauth|pubkeyauth'
sudo -u www-data crontab -l
sudo certbot certificates
cat /etc/letsencrypt/renewal/app.robrands.ro.conf
```

16. Glosar (pentru cititori non-tehnici)

Termen	Explicație simplă
VPS	Un calculator închiriat „în nor”, pornit non-stop, pe care rulează aplicația.
SSH	Modul securizat de a te conecta de la distanță la server, ca să dai comenzi.
nginx	„Recepționarul”, care primește vizitatorii și îi îndrumă spre aplicație.
Gunicorn	Motorul care rulează efectiv codul aplicației.
Flask	Tehnologia (limbajul Python) în care e scrisă aplicația.
SQLite	Baza de date — un singur fișier cu toate datele.
systemd	Mecanismul care pornește automat aplicația la repornirea serverului.
HTTPS / SSL / certificat	„Lacătul”, din browser — comunicarea criptată cu site-ul.
Let's Encrypt / Certbot	Serviciul gratuit care emite și reînnoiește lacătul.
Firewall (ufw)	„Poarta”, care lasă să intre doar traficul permis.
fail2ban	„Paznicul”, care blochează cine încearcă parole greșite repetat.
CI/CD / GitHub Actions	„Banda de montaj”, care testează și publică automat noile versiuni.
Deploy / implementare	Publicarea unei noi versiuni a aplicației pe server.
Backup	Copie de siguranță a datelor, pentru recuperare.
sudo	Permisiunea de „administrator”, pentru comenzi cu impact ridicat.
.env	Fișierul cu setări și parole secrete ale aplicației.
OpenClaw	Componenta care alimentează asistentul AI din aplicație.

17. Contact și suport

Pentru asistență tehnică sau raportarea unor probleme neacoperite în acest manual, contactați echipa de suport:

Email: suport@torblogistic.ro

Telefon: +40 XXX XXX XXX (L–V, 09:00–17:00)

Furnizor hosting: cyberfolks.ro — date de logare în seiful de secrete

© 2026 Torb Logistic SRL. Document intern — uz exclusiv al angajaților autorizați.